UNIT 6

SERVICE ORIENTED ARCHITECTURE IN CLOUD COMPUTING

6.1. Introduction to Processing Pipelines, Batch Processing Systems in Cloud Web Applications:

- Data must be processed before it is consumed.
- By definition, a data pipeline represents the flow of data between two or more systems.
- It is a set of instructions that determine how and when to move data between these systems.

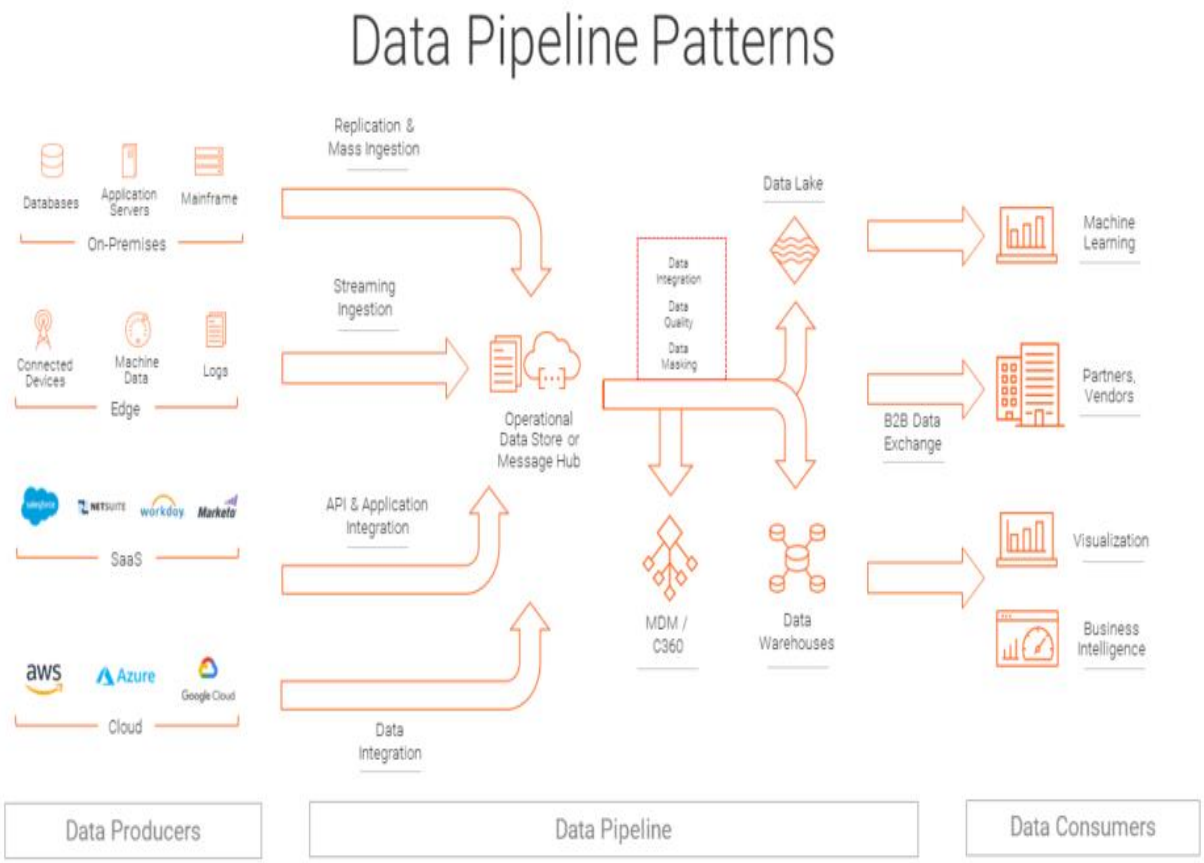
There are many data processing pipelines. One may:

- "Integrate" data from multiple sources
- Perform data quality checks or standardize data
- Apply data security-related transformations, which include masking, anonymizing, or encryption
- Match, merge, master, and do entity resolution
- Share data with partners and customers in the required format

Consumers or "targets" of data pipelines may include:

- Data warehouses like Redshift, Snowflake, SQL data warehouses, or Teradata
- Reporting tools like Tableau or Power BI
- Another application in the case of application integration or application migration
- Data lakes on Amazon S3, Microsoft ADLS, or Hadoop – typically for further exploration
- Artificial intelligence algorithms
- Temporary repositories or publish/subscribe queues like Kafka for consumption by a downstream data pipeline

The following architecture shows the data pipeline processing



6.1.1 **Batch processing**

- ❖ It is a method of running high-volume, repetitive data jobs.
- ❖ The batch method allows users to process data when computing resources are available, and with little or no user interaction.
- ❖ Batch processing plays a critical role in helping companies and organizations manage large amounts of data efficiently.
- ❖ It is especially suited for handling frequent, repetitive tasks such as accounting processes.
- ❖ In every industry and for every job, the basics of batch processing remain the same.

The essential parameters include:

- who is submitting the job?
- which program will run the location of the input and outputs?
- when the job should be run.

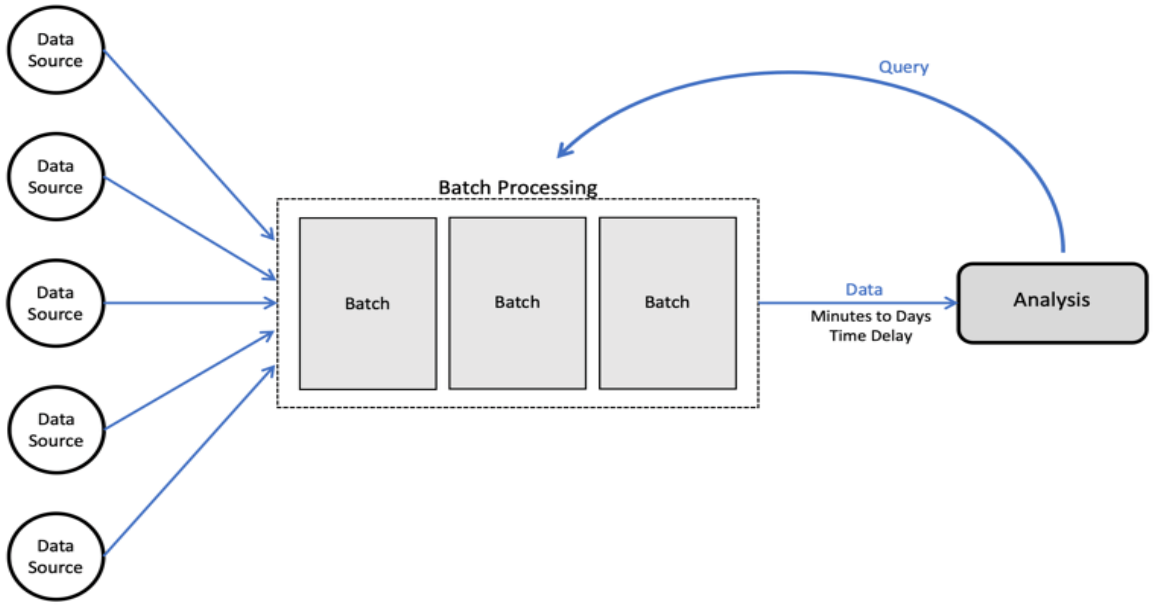


Fig: Detail of Batch processing

6.2. Cloud Computing Architecture

Architecture of cloud computing is the combination of both **SOA (Service Oriented Architecture) and EDA (Event Driven Architecture).**

Frontend

Backend

Please refer UNIT 1 for the explanation of this architecture

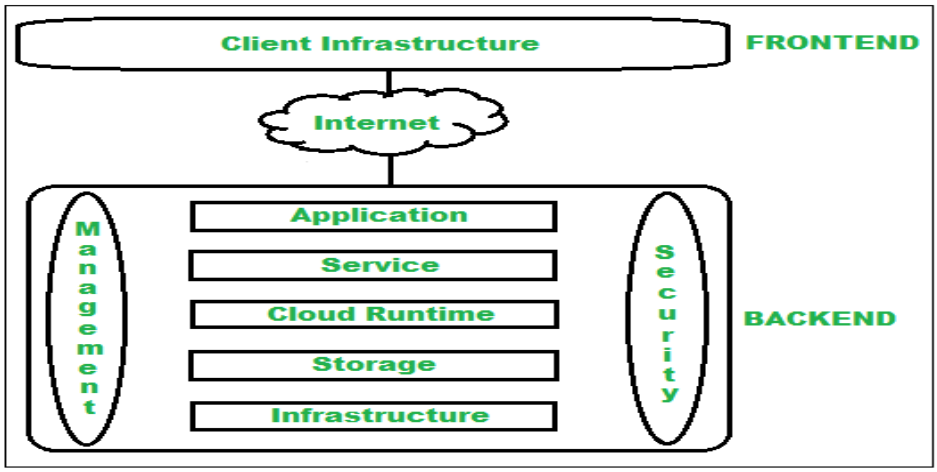


Fig: Cloud Computing Architecture based on SOA and EDA

6.3. Work Flow in Cloud Computing:

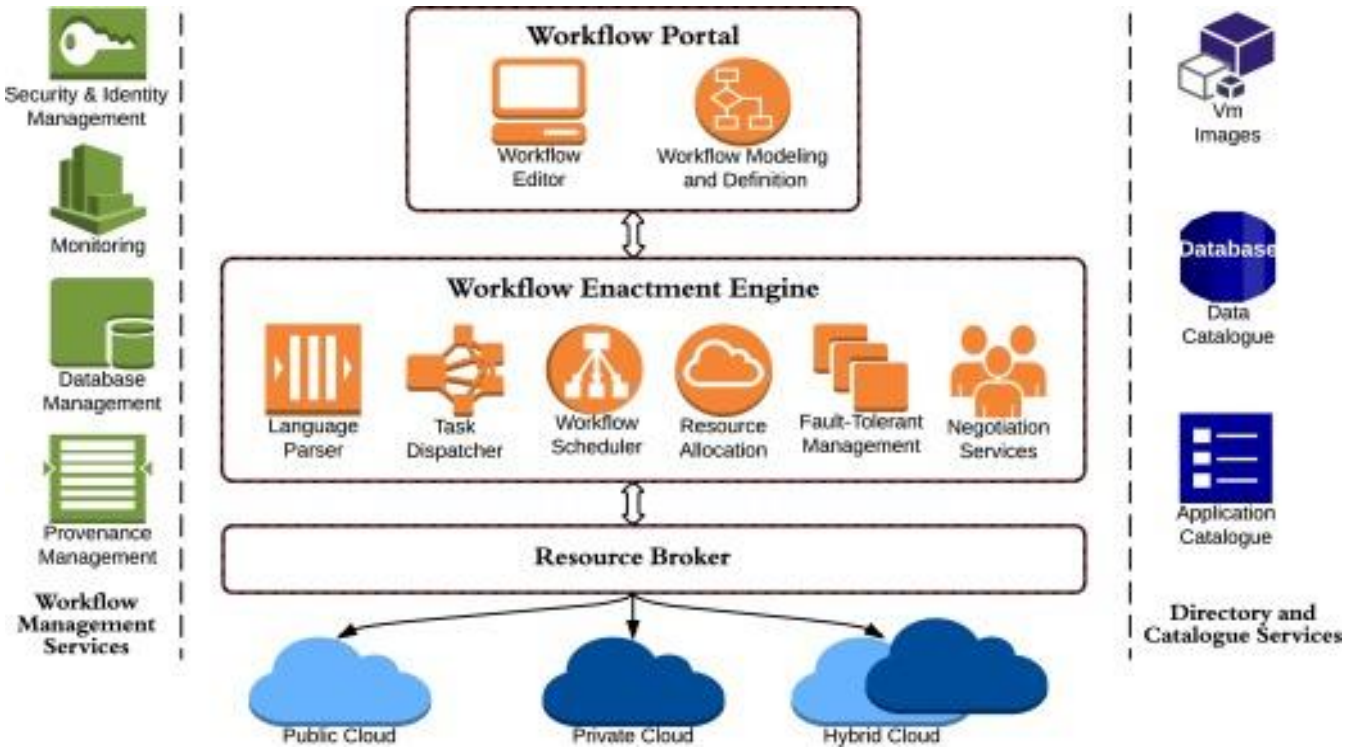


Fig: A typical Work Flow in Cloud

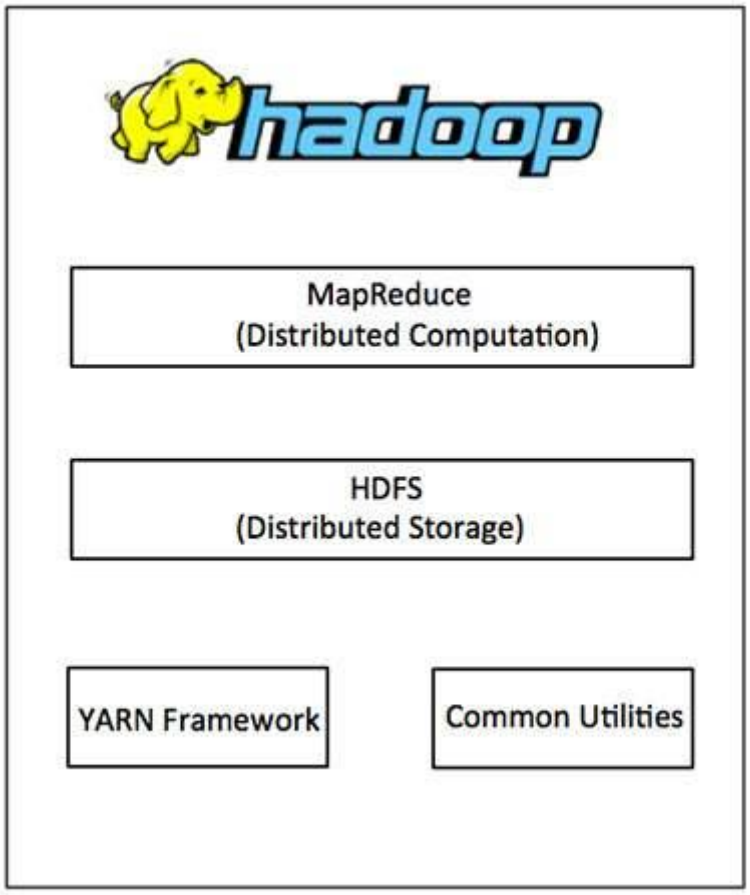
6.4. Hadoop:

- Hadoop is an Apache open-source framework written in java that allows distributed processing of large datasets across clusters of computers using simple programming models.
- The Hadoop framework application works in an environment that provides distributed storage and computation across clusters of computers.
- Hadoop is designed to scale up from single server to thousands of machines, each offering local computation and storage.

Hadoop Architecture

At its core, Hadoop has two major layers namely −

- Processing/Computation layer (MapReduce), and
- Storage layer (Hadoop Distributed File System).



MapReduce

- MapReduce is a parallel programming model for writing distributed applications devised at Google for efficient processing of large amounts of data (multi-terabyte data-sets), on large clusters (thousands of nodes) of commodity hardware in a reliable, fault-tolerant manner.
- The MapReduce program runs on Hadoop which is an Apache open-source framework.

Hadoop Distributed File System

- The Hadoop Distributed File System (HDFS) is based on the Google File System (GFS) and provides a distributed file system that is designed to run on commodity hardware.
- It has many similarities with existing distributed file systems. However, the differences from other distributed file systems are significant.
- It is highly fault-tolerant and is designed to be deployed on low-cost hardware.
- It provides high throughput access to application data and is suitable for applications having large datasets.

Apart from the above-mentioned two core components, Hadoop framework also includes the following two modules −

Hadoop Common − These are Java libraries and utilities required by other Hadoop modules.

Hadoop YARN − This is a framework for job scheduling and cluster resource management.

Working of Hadoop

- It is quite expensive to build bigger servers with heavy configurations that handle large scale processing, but as an alternative, you can tie together many commodity computers with single-CPU, as a single functional distributed system and practically, the clustered machines can read the dataset in parallel and provide a much higher throughput.
- Moreover, it is cheaper than one high-end server. So this is the first motivational factor behind using Hadoop that it runs across clustered and low-cost machines.
- Hadoop runs code across a cluster of computers.

This process includes the following core tasks that Hadoop performs –

- Data is initially divided into directories and files. Files are divided into uniform sized blocks of 128M and 64M (preferably 128M).
- These files are then distributed across various cluster nodes for further processing.
- HDFS, being on top of the local file system, supervises the processing.
- Blocks are replicated for handling hardware failure.
- Checking that the code was executed successfully.
- Performing the sort that takes place between the map and reduce stages.
- Sending the sorted data to a certain computer.
- Writing the debugging logs for each job.

Advantages of Hadoop

- Hadoop framework allows the user to quickly write and test distributed systems.
- It is efficient, and it automatically distributes the data and work across the machines and in turn, utilizes the underlying parallelism of the CPU cores.
- Hadoop does not rely on hardware to provide fault-tolerance and high availability (FTHA), rather Hadoop library itself has been designed to detect and handle failures at the application layer.
- Servers can be added or removed from the cluster dynamically and Hadoop continues to operate without interruption.
- Another big advantage of Hadoop is that apart from being open source, it is compatible on all the platforms since it is Java based.

6.5. HDFS (Hadoop Distributed File System):

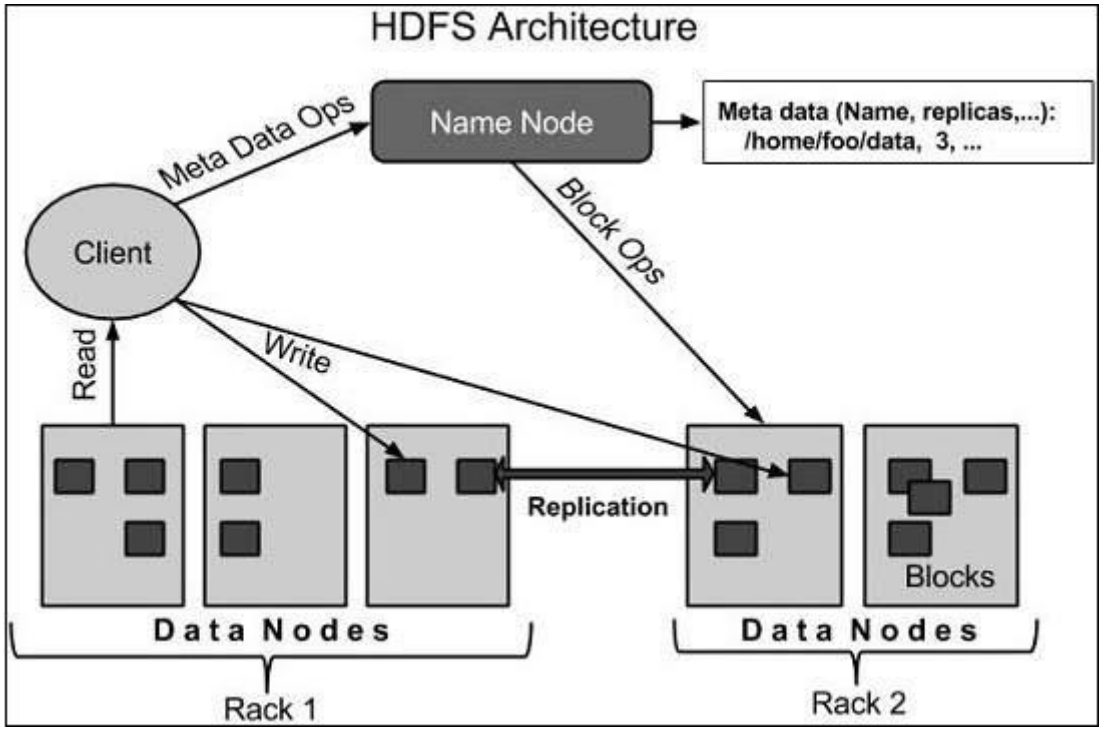
- Hadoop File System was developed using distributed file system design.
- It runs on commodity hardware.
- Unlike other distributed systems, HDFS is highly fault tolerant and designed using low-cost hardware.
- HDFS holds very large amount of data and provides easier access.
- To store such huge data, the files are stored across multiple machines.
- These files are stored in redundant fashion to rescue the system from possible data losses in case of failure.
- HDFS also makes applications available to parallel processing.

Features of HDFS

- It is suitable for the distributed storage and processing.
- Hadoop provides a command interface to interact with HDFS.
- The built-in servers of namenode and datanode help users to easily check the status of cluster.
- Streaming access to file system data.
- HDFS provides file permissions and authentication.

HDFS Architecture

Given below is the architecture of a Hadoop File System.



HDFS follows the master-slave architecture and it has the following elements.

Namenode

- The namenode is the commodity hardware that contains the GNU/Linux operating system and the namenode software.
- It is a software that can be run on commodity hardware.
- The system having the namenode acts as the master server and it does the following tasks –
  Manages the file system namespace.
  Regulates client's access to files.
  It also executes file system operations such as renaming, closing, and opening files and directories.

Datanode

- The datanode is a commodity hardware having the GNU/Linux operating system and datanode software.
- For every node (Commodity hardware/System) in a cluster, there will be a datanode.
- These nodes manage the data storage of their system.
  Datanodes perform read-write operations on the file systems, as per client request.
  They also perform operations such as block creation, deletion, and replication according to the instructions of the namenode.

Block

- Generally the user data is stored in the files of HDFS.
- The file in a file system will be divided into one or more segments and/or stored in individual data nodes.
- These file segments are called as blocks. In other words, the minimum amount of data that HDFS can read or write is called a Block.
- The default block size is 64MB, but it can be increased as per the need to change in HDFS configuration.

Goals of HDFS

- **Fault detection and recovery** − Since HDFS includes a large number of commodity hardware, failure of components is frequent. Therefore, HDFS should have mechanisms for quick and automatic fault detection and recovery.
- **Huge datasets** − HDFS should have hundreds of nodes per cluster to manage the applications having huge datasets.
- **Hardware at data** − A requested task can be done efficiently, when the computation takes place near the data. Especially where huge datasets are involved, it reduces the network traffic and increases the throughput.

6.6. MapReduce:

- MapReduce is a framework using which we can write applications to process huge amounts of data, in parallel, on large clusters of commodity hardware in a reliable manner.
- MapReduce is a processing technique and a program model for distributed computing based on java.
- The MapReduce algorithm contains two important tasks, namely Map and Reduce.
- Map takes a set of data and converts it into another set of data, where individual elements are broken down into tuples (key/value pairs).
- Secondly, reduce task, which takes the output from a map as an input and combines those data tuples into a smaller set of tuples.
- As the sequence of the name MapReduce implies, the reduce task is always performed after the map job.

Working of MapReduce:

- Generally MapReduce paradigm is based on sending the computer to where the data resides!
- MapReduce program executes in three stages, namely map stage, shuffle stage, and reduce stage.

Map stage − The map or mapper's job is to process the input data.

- Generally, the input data is in the form of file or directory and is stored in the Hadoop file system (HDFS).
- The input file is passed to the mapper function line by line.
- The mapper processes the data and creates several small chunks of data.

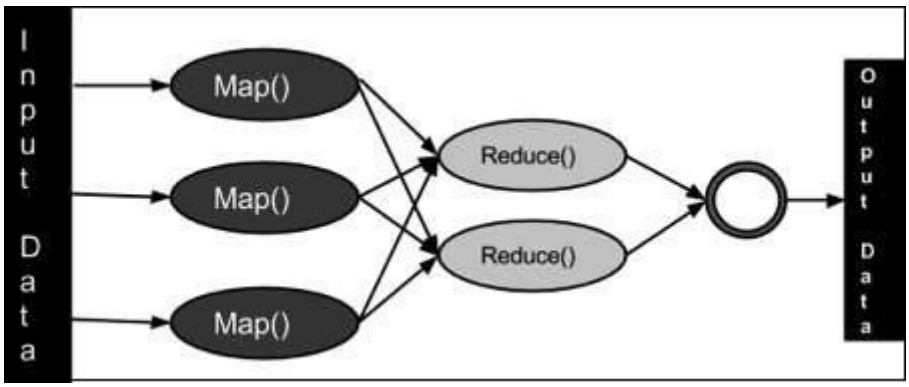
Reduce stage − This stage is the combination of the Shuffle stage and the Reduce stage.

- The Reducer's job is to process the data that comes from the mapper.
- After processing, it produces a new set of output, which will be stored in the HDFS.
- During a MapReduce job, Hadoop sends the Map and Reduce tasks to the appropriate servers in the cluster.

The framework manages all the details of data-passing such as issuing tasks, verifying task completion, and copying data around the cluster between the nodes.

Most of the computing takes place on nodes with data on local disks that reduces the network traffic.

After completion of the given tasks, the cluster collects and reduces the data to form an appropriate result, and sends it back to the Hadoop server.



Architecture:

MapReduce Architecture explained in detail

- One map task is created for each split which then executes map function for each record in the split.
- It is always beneficial to have multiple splits because the time taken to process a split is small as compared to the time taken for processing of the whole input. When the

splits are smaller, the processing is better to load balanced since we are processing the splits in parallel.

- However, it is also not desirable to have splits too small in size. When splits are too small, the overload of managing the splits and map task creation begins to dominate the total job execution time.
- For most jobs, it is better to make a split size equal to the size of an HDFS block (which is 64 MB, by default).
- Execution of map tasks results into writing output to a local disk on the respective node and not to HDFS.
- Reason for choosing local disk over HDFS is, to avoid replication which takes place in case of HDFS store operation.
- Map output is intermediate output which is processed by reduce tasks to produce the final output.
- Once the job is complete, the map output can be thrown away. So, storing it in HDFS with replication becomes overkill.
- In the event of node failure, before the map output is consumed by the reduce task, Hadoop reruns the map task on another node and re-creates the map output.
- Reduce task doesn't work on the concept of data locality. An output of every map task is fed to the reduce task. Map output is transferred to the machine where reduce task is running.
- On this machine, the output is merged and then passed to the user-defined reduce function.
- Unlike the map output, reduce output is stored in HDFS (the first replica is stored on the local node and other replicas are stored on off-rack nodes). So, writing the reduce output

Benefits:

- **Scalability.** Businesses can process petabytes of data stored in the Hadoop Distributed File System (HDFS).
- **Flexibility**. Hadoop enables easier access to multiple sources of data and multiple types of data.
- **Speed**. With parallel processing and minimal data movement, Hadoop offers fast processing of massive amounts of data.
- **Simple**. Developers can write code in a choice of languages, including Java, C++ and Python.
- **Security and Authentication**: -In this regard, MapReduce works with HDFS and HBase security that allows only approved users to operate on data stored in the system.

6.7. SOA (Service Oriented Architecture)

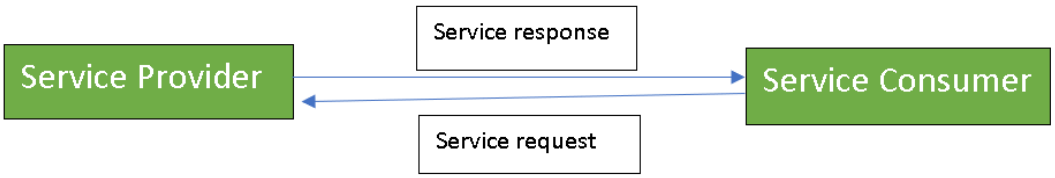
- Service-Oriented Architecture (SOA) is a stage in the evolution of application development and/or integration.
- It defines a way to make software components reusable using the interfaces.
- Formally, SOA is an architectural approach in which applications make use of services available in the network.

- In this architecture, services are provided to form applications, through a network call over the internet.
- It uses common communication standards to speed up and streamline the service integrations in applications.
- Each service in SOA is a complete business function in itself.
- The services are published in such a way that it makes it easy for the developers to assemble their apps using those services. Note that SOA is different from microservice architecture.
- SOA allows users to combine a large number of facilities from existing services to form applications.
- SOA encompasses a set of design principles that structure system development and provide means for integrating components into a coherent and decentralized system.
- SOA-based computing packages functionalities into a set of interoperable services, which can be integrated into different software systems belonging to separate business domains.

There are two major roles within Service-oriented Architecture:

Service provider: The service provider is the maintainer of the service and the organization that makes available one or more services for others to use. To advertise services, the provider can publish them in a registry, together with a service contract that specifies the nature of the service, how to use it, the requirements for the service, and the fees charged.

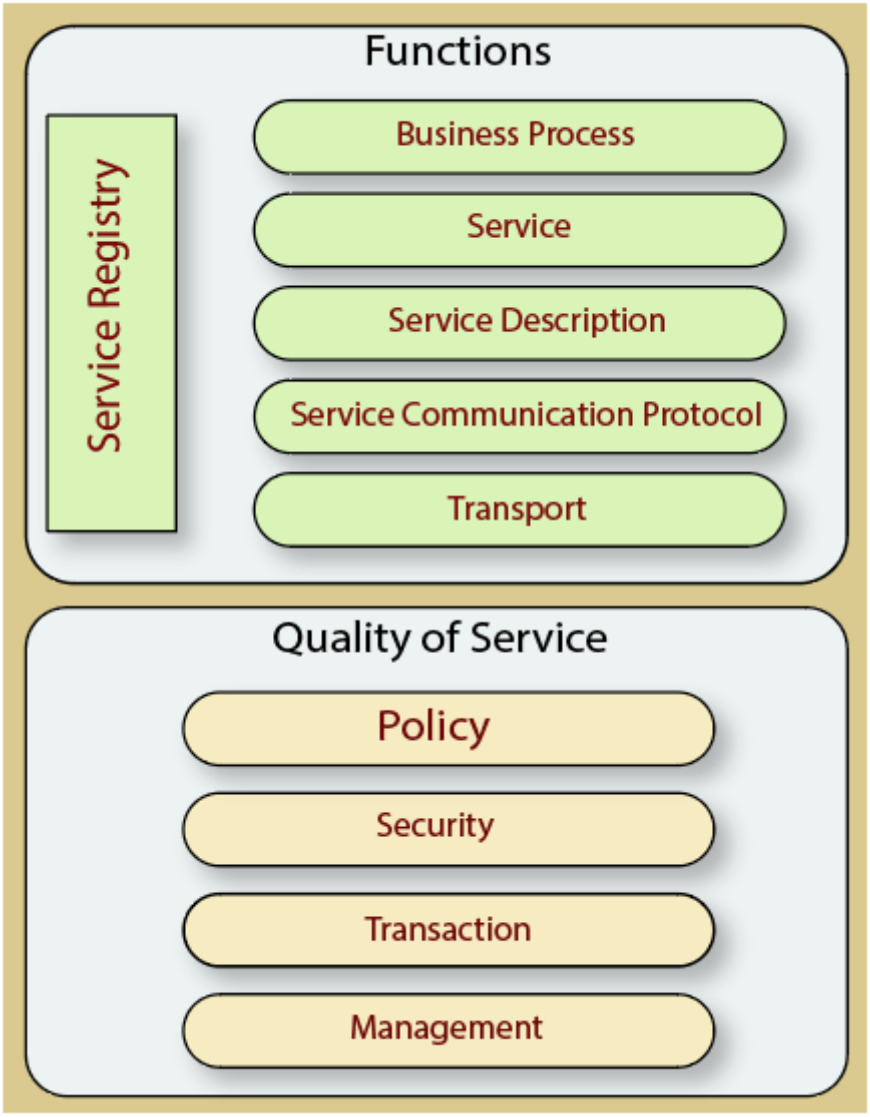
Service consumer: The service consumer can locate the service metadata in the registry and develop the required client components to bind and use the service.



- Services might aggregate information and data retrieved from other services or create workflows of services to satisfy the request of a given service consumer.
- This practice is known as service orchestration
- Another important interaction pattern is service choreography, which is the coordinated interaction of services without a single point of control.

Components of service-oriented architecture

The service-oriented architecture stack can be categorized into two parts - **functional aspects and quality of service aspects.**



Functional aspects

The functional aspect contains:

Transport - It transports the service requests from the service consumer to the service provider and service responses from the service provider to the service consumer.

Service Communication Protocol - It allows the service provider and the service consumer to communicate with each other.

Service Description - It describes the service and data required to invoke it.

Service - It is an actual service.

Business Process - It represents the group of services called in a particular sequence associated with the particular rules to meet the business requirements.

Service Registry - It contains the description of data which is used by service providers to publish their services.

Quality of Service aspects

The quality-of-service aspects contains:

Policy - It represents the set of protocols according to which a service provider make and provide the services to consumers.

Security - It represents the set of protocols required for identification and authorization.

Transaction - It provides the surety of consistent result. This means, if we use the group of services to complete a business function, either all must complete or none of the complete.

Management - It defines the set of attributes used to manage the services.

Guiding Principles of SOA:

Standardized service contract: Specified through one or more service description documents.

Loose coupling: Services are designed as self-contained components, maintain relationships that minimize dependencies on other services.

Abstraction: A service is completely defined by service contracts and description documents. They hide their logic, which is encapsulated within their implementation.

Reusability: Designed as components, services can be reused more effectively, thus reducing development time and the associated costs.

Autonomy: Services have control over the logic they encapsulate and, from a service consumer point of view, there is no need to know about their implementation.

Discoverability: Services are defined by description documents that constitute supplemental metadata through which they can be effectively discovered. Service discovery provides an effective means for utilizing third-party resources.

Composability: Using services as building blocks, sophisticated and complex operations can be implemented. Service orchestration and choreography provide a solid support for composing services and achieving business goals.

Advantages of SOA:

Service reusability: In SOA, applications are made from existing services. Thus, services can be reused to make many applications.

Easy maintenance: As services are independent of each other they can be updated and modified easily without affecting other services.

Platform independent: SOA allows making a complex application by combining services picked from different sources, independent of the platform.

Availability: SOA facilities are easily available to anyone on request.

Reliability: SOA applications are more reliable because it is easy to debug small services rather than huge codes

Scalability: Services can run on different servers within an environment, this increases scalability

Disadvantages of SOA:

High overhead: A validation of input parameters of services is done whenever services interact this decreases performance as it increases load and response time.

High investment: A huge initial investment is required for SOA.

Complex service management: When services interact, they exchange messages to tasks. the number of messages may go in millions. It becomes a cumbersome task to handle a large number of messages.

Practical applications of SOA:

SOA is used in many ways around us whether it is mentioned or not.

- SOA infrastructure is used by many armies and air forces to deploy situational awareness systems.
- SOA is used to improve healthcare delivery.
- Nowadays many apps are games and they use inbuilt functions to run.
  For example, an app might need GPS so it uses the inbuilt GPS functions of the device.
  This is SOA in mobile solutions.
- ❖ SOA helps maintain museums a virtualized storage pool for their information and content.